

## Assignment-3

Name - Priyanshi Patel

Roll no. - 2401010268

Section - A

Program - B.Tech CSE

Course - Data Base Management System

Course code - ENCS204

Submitted to:- Manvi Kajal

### # Section - A

1) What is SQL? Why is it important in DBMS?

Ans:- → SQL (Structured Query language) is a standard language used to interact with relational databases.

→ It is specifically designed to manage & manipulate structured data stored in tables (relations).

→ In a database, data is organized in rows & columns, & SQL provides commands to perform operations such as creating tables, inserting data, retrieving data, updating records & deleting data.

→ Why SQL is important in DBMS:

1) Data Retrieval (Querying Data)

→ SQL allows users to retrieve specific data efficiently using conditions.

```
SELECT name, marks  
FROM students
```

```
WHERE marks > 75;
```

→ This query returns only those students who scored more than 75 marks.

## 2) Data Manipulation :-

→ SQL helps modify existing data in the database.

UPDATE students

SET marks = 85

WHERE id = 2;

## 3) Data Definition (Structure Creation)

→ SQL is used to define the structure of the database.

CREATE TABLE students (

id INT PRIMARY KEY,

name VARCHAR(50),

marks INT

);

## 4) Data Integrity & Constraints

→ SQL ensures accuracy using constraints

- PRIMARY KEY → unique identification
- FOREIGN KEY → maintains relationships
- NOT NULL → prevents empty values

## 5) Security & Access Control

→ SQL allows assigning permissions:

GRANT SELECT ON students TO user1;

## 6) Supports Complex Operations:

→ SQL supports:

- Joins
- Subqueries
- Aggregations

Q) What is the difference b/w DDL & DML?

Ans:- DDL (Data Definition language):

→ Common Commands:

- CREATE
- ALTER
- DROP
- TRUNCATE

→ Example:-

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

→ This creates a table structure.

→ DML (Data Manipulation Language):

→ DML is use to manipulate the data stored in tables:

Common Commands:

- INSERT
- UPDATE
- DELETE
- SELECT

Ex:- INSERT INTO students VALUES (1, 'Riya');

→ This adds data into the table.

| Feature  | DDL              | DML              |
|----------|------------------|------------------|
| Purpose  | Define structure | Manipulates data |
| Scope    | Schema level     | Data level       |
| Commands | CREATE, ALTER    | INSERT, UPDATE   |
| Rollback | Not possible     | Possible         |
| Effect   | Permanent        | can be reversed  |

→ DDL operations are auto-committed.

→ DML operations can be controlled using transaction (COMMIT ROLLBACK)

Ques: 3 what is Join? Explain its types. (4)

Ans: → A JOIN is used to combine data from two or more table based on related column.

→ In relational databases, data is stored in multiple tables to reduce redundancy. JOIN helps retrieve related data in a single query.

Example:-

| Students |      |           | Courses   |             |
|----------|------|-----------|-----------|-------------|
| id       | name | course-id | course-id | course-name |
| 1        | A    | 101       | 101       | CS          |
| 2        | B    | 102       | 102       | IT          |
| 3        | C    | 103       |           |             |

#### 1) INNER JOIN

→ Returns only matching records.

```
SELECT s.name, c.course_name
```

```
FROM students s
```

```
INNER JOIN courses c
```

```
ON s.course-id = c.course-id;
```

→ output: only A & B (C is excluded because no match).

#### 2) LEFT JOIN

→ Returns all records from left table.

```
SELECT s.name, c.course_name
```

```
FROM students s
```

```
LEFT JOIN courses c
```

```
ON s.course-id = c.course-id;
```

→ output: A, B & C (C will have NULL course)

#### 3) RIGHT JOIN

→ Returns all records from right table.

#### 4) FULL JOIN

→ Returns all records from both tables.

- JOIN is based on primary key - foreign key relationship
- Helps maintain data normalization.

4) What is GROUP BY and Having clause?

Ans:- GROUP BY: Used to group rows that have the same values.

Ex:- SELECT department, COUNT(\*)  
FROM students

GROUP BY department;

→ Groups students by department & counts them

→ HAVING:- used to filter grouped data

SELECT department, COUNT(\*)

FROM students

GROUP BY department

HAVING COUNT(\*) > 2;

→ Shows only departments with more than 2 students.

Example:-

| name | department |
|------|------------|
| A    | CS         |
| B    | CS         |
| C    | IT         |
| D    | IT         |
| E    | IT         |

→ Result:

- CS → 2
- IT → 3

→ Using Aggregate Functions:

SELECT department, AVG(marks)

FROM students

GROUP BY departments;

→ Group By works with:

- COUNT()

- SUM()

- AVG()

→ Having is used after grouping.

→ Difference with WHERE:  
 -- Before grouping  
 WHERE marks > 50

-- After grouping

HAVING AVG(marks) > 70

Ques:- What is subquery?

Ans:- A subquery is a query written inside another SQL query. It is used to perform complex operations step by step.

Ex:- ① SELECT name  
 FROM students

WHERE marks > (SELECT AVG(marks) FROM students);

→ Find students scoring above average.

Ex:- ② SELECT name  
 FROM students

WHERE course-id IN (

SELECT course-id FROM courses WHERE course-name = 'CS'  
 );

→ Types of subqueries:-

- 1) Single-row subquery
- 2) Multi-row subquery
- 3) Correlated subquery

→ Correlated example:-

SELECT s1.name

FROM students s1

WHERE marks > (

SELECT AVG(marks)

FROM students s2

WHERE s1.department = s2.department

);

# Advantages:-

- Breaks complex queries into simpler parts
- Improves readability
- Useful for nested conditions.

Ques:- 6 What is relational algebra?

Ans:- Relational algebra is a procedural query language used as the theoretical foundation of relational database.

→ Operations:-

| operation  | symbol    | Meaning          |
|------------|-----------|------------------|
| Selection  | $\sigma$  | Select rows      |
| Projection | $\pi$     | Select columns   |
| Join       | $\bowtie$ | combine tables   |
| Union      | $\cup$    | Merge results    |
| Difference | $-$       | subtract results |

Example:-

- $\sigma(\text{marks} > 80) \rightarrow$  select rows
- $\pi(\text{name})$  select column

- # Importance:-
- Forms base of SQL
  - Helps in query optimization
  - Used in database design

Ques:- 7 Difference b/w WHERE and HAVING

Ans:- WHERE Clause:

- Filters rows before grouping
- Cannot use aggregate functions

SELECT \* FROM students WHERE marks > 70;

→ HAVING Clause:-

- Filters grouped data
- Works with aggregate functions

SELECT department, COUNT(\*)  
FROM students  
GROUP BY department  
HAVING COUNT(\*) > 2;

→ Combined Example :-

```
SELECT department, AVG(marks)
FROM students
WHERE marks > 50
GROUP BY department
HAVING AVG(marks) > 70;
```

→ Key Differences :-

| Feature              | where       | Having  |
|----------------------|-------------|---------|
| Works on             | Rows        | Groups  |
| Used before GROUP BY | Yes         | No      |
| Aggregate Functions  | Not allowed | allowed |

- WHERE improves performance by filtering early
- HAVING is used when conditions depend on grouped results.

### # Section :- B

Ques:- 1 Write an SQL query to retrieve all students with marks > 75.

Ans:- 

```
SELECT * FROM students
WHERE marks > 75;
```

- SELECT \* → retrieves all columns from the table
- FROM students → specifies the table
- WHERE marks > 75 → filters only those students whose marks are greater than 75
- This condition ensures only high-performing students are selected.

→ Example :-

| id | name | marks |
|----|------|-------|
| 1  | A    | 80    |
| 2  | B    | 60    |
| 3  | C    | 90    |

→ output:

- A(80)
- C(90)

- The WHERE clause is used for row-level filtering.
- It improves performance by reducing unnecessary data.
- You can also combine conditions:

Ques:- 2 Write a query using JOIN to combine Student & Course Table.

Ans: SELECT s.name, c.course-name  
FROM Students s  
INNER JOIN courses c  
ON s.course-id = c.course-id;

- Students s → alias for student table
- Courses c → alias for course table
- INNER JOIN → combines rows where condition matches
- ON ~~s.name~~ s.course-id = c.course-id → join condition
- This query retrieves students names along with their course names.

→ Example:-

| Students |      |           | Courses   |             |
|----------|------|-----------|-----------|-------------|
| id       | Name | course-id | Course-id | course-name |
| 1        | A    | 101       | 101       | CS          |
| 2        | B    | 102       | 102       | IT          |
| 3        | C    | 103       |           |             |

→ output:-

| name | course-name |
|------|-------------|
| A    | CS          |
| B    | IT          |

→ C is excluded because no matching course.

Ques:- 3 Write a query using GROUP BY to count students per department.

Ans: SELECT department, COUNT(\*) AS total\_students  
FROM students  
GROUP BY department;

- department → grouping column
- COUNT(\*) → counts number of students
- GROUP BY department → groups rows by department.

→ Example:-

| name | department |
|------|------------|
| A    | CS         |
| B    | CS         |
| C    | IT         |
| D    | IT         |
| E    | IT         |

→ Output:-

| department | total-students |
|------------|----------------|
| CS         | 2              |
| IT         | 3              |

### #Section :- c

Ques:- A Company database stores employee & department data, write a query to find employees in specific department.

Ans:-

```

SELECT e.name, d.department_name
FROM employee e
JOIN departments d
ON e.dept_id = d.dept_id
WHERE d.department_name = 'HR';
    
```

- employee e → employee table (alias used for simplicity)
- departments d → department table
- JOIN → combines both tables using a common column (dept\_id)
- WHERE → filters only employees working in the HR departments

→ Example:-

| Employees |      |         | dept-id | department_name |
|-----------|------|---------|---------|-----------------|
| id        | name | dept-id | 10      | HR              |
| 1         | A    | 10      | 20      | IT              |
| 2         | B    | 20      |         |                 |
| 3         | C    | 30      |         |                 |

→ Output:-

| name | department-name |
|------|-----------------|
| A    | HR              |
| C    | HR              |

→ why JOIN is used Here:-

→ employee table does not store department name directly

→ Department name is stored in another table.

→ JOIN helps combine related table.

→ This is an example of relational database normalization

→ Improves data consistency (no duplication of department names).

Ques:- write An e-commerce system stores orders. Retrieve total sales Per customer.

Ans:-  
SELECT customer-id, SUM(amount) AS total-sales  
FROM orders  
GROUP BY customer-id;

→ customer-id → identifies each customer

→ SUM(amount) → calculates total purchase value

→ GROUP BY customer-id → group all orders by each customer

→ Example:-

| order-id | customer-id | amount |
|----------|-------------|--------|
| 1        | 101         | 500    |
| 2        | 101         | 300    |
| 3        | 102         | 700    |

→ output:-

| customer-id | total-sales |
|-------------|-------------|
| 101         | 800         |
| 102         | 700         |

- Each customer may have multiple orders
- GROUP BY combines them into one summary row

Ques:- 3 A university database needs top 3 students. Write a query.

Ans:- SELECT \* FROM students

ORDER BY marks DESC

LIMIT 3;

- ORDER BY marks DESC → sorts students from highest to lowest marks
- LIMIT 3 → selects only top 3 students

→ Example:-

| name | marks |
|------|-------|
| A    | 85    |
| B    | 92    |
| C    | 78    |
| D    | 95    |

→ output:-

| name | marks |
|------|-------|
| D    | 95    |
| B    | 92    |
| A    | 85    |

# Section D :-

Ques:- 1 Write an SQL query for any real-life system & explain the logic

Ans:- E-commerce System - Best Selling Products:

SELECT product-id, SUM(quantity) AS total-sold  
FROM sales

GROUP BY product-id

ORDER BY total-sold DESC;

→ This query is used in an e-commerce system to identify the most sold products

- product-id → identifies each product
- SUM(quantity) → calculates total units sold for each product
- GROUP BY product-id → groups all sales records product-wise.
- ORDER BY total-sold DESC → sorts products from highest to lowest sales

→ Example :-

| sale-id | product-id | quantity |
|---------|------------|----------|
| 1       | 101        | 2        |
| 2       | 101        | 3        |
| 3       | 102        | 5        |

# Output

| product-id | total-sold |
|------------|------------|
| 101        | 5          |
| 102        | 5          |

Ques:- 2 Write a JOIN query & mention one use case

Ans:- SELECT o.order-id, c.customer-name, o.amount

FROM orders o

JOIN customer c

ON o.customer-id = c.customer-id;

→ combines data from orders & customers tables

→ customer-id is the common column

→ Returns order details along with customer name

→ Example

| order-id | customer-id | amount |
|----------|-------------|--------|
| 1        | 101         | 500    |
| 2        | 102         | 300    |

#customers

| customer-id | customer-name |
|-------------|---------------|
| 101         | Rahul         |
| 102         | Priya         |

→ output:-

| order-id | customer-name | amount |
|----------|---------------|--------|
| 1        | Rahul         | 500    |
| 2        | Priya         | 300    |

→ Used in e-commerce platform to display:

- Customer name
- Order details
- Payment information

→ JOIN helps maintain data normalization

→ Avoids duplication of customer data in multiple tables.

Ques:- 3 Write a GROUP BY query and explain its output:

Ans:- Average Salary per Department:-

```
SELECT department, AVG(salary) AS avg-salary
FROM employees
```

GROUP BY department;

→ department → grouping column

→ AVG(salary) → calculates average salary

→ GROUP BY department → groups employees by department

→ Example

15

| name | department | salary |
|------|------------|--------|
| A    | HR         | 30000  |
| B    | HR         | 40000  |
| C    | IT         | 80000  |
| D    | IT         | 60000  |

→ output:

| department | avg. salary |
|------------|-------------|
| HR         | 35000       |
| IT         | 55000       |

→ Helps organizations:

- Compare salary distribution
- Analyze department performance
- Make budget decisions